



Garia Green

Technical WhitePaper

KtheCoder

Gaia Green

Abstract

What is a game without a terrain? How does one go about creating engaging and alive environments? Environments that can speak to the qualities of the games vision. Where the earth lays itself into existence to breathe its calming beauty into its representation.

Gaia Green sets out to achieve a simple goal of combining re-usability into usability. Attaining a Procedural Generation of Terrain with a simplistic tile based system that intertwines with the randomness of a simple noise generation.

Introduction

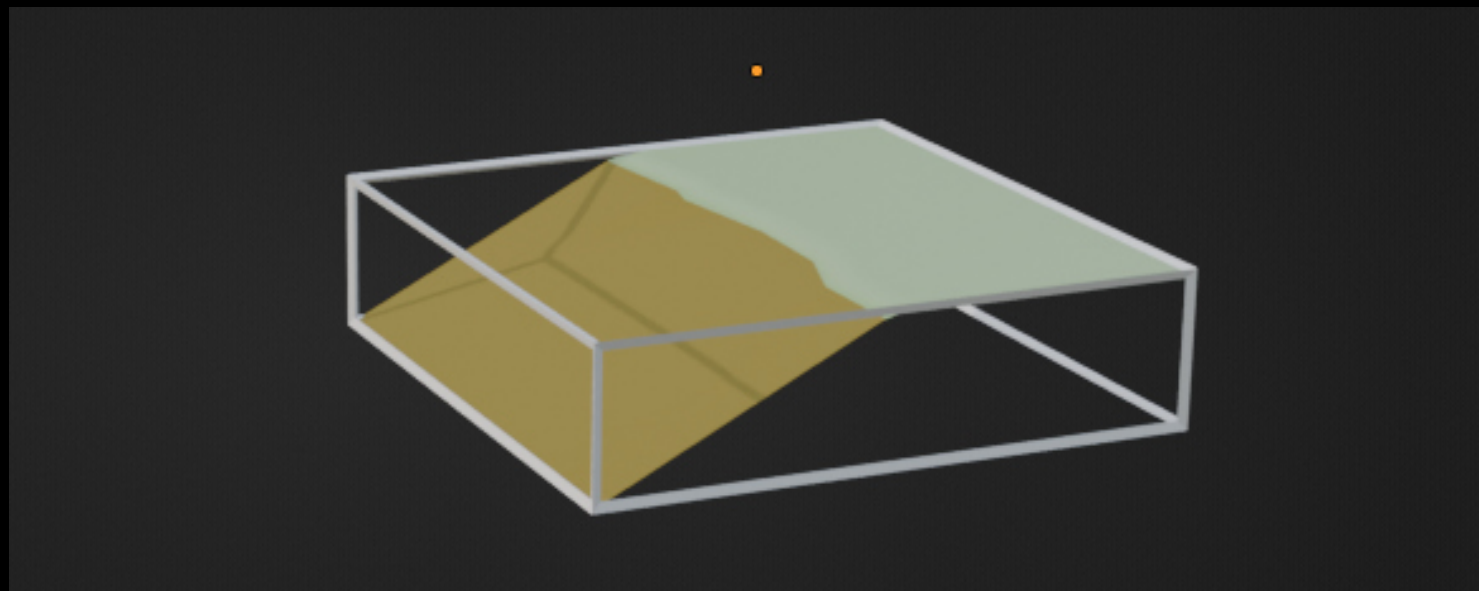
Welcome, wanderer of the digital realms, to Gaia Green; an open source terrain generation odyssey for Godot that harnesses the primal chaos of FastNoiseLite to sculpt breathtakingly simple 3D landscapes. Weaving strands of mathematical entropy into tactile topographies that ripple with life and logic.

Crafted for thinkers who crave both algorithmic elegance and aesthetic depth. It treats terrain not as mere geometry, but as a narrative canvas. Each cliff, ramp, and valley a verse in a spatial poem.

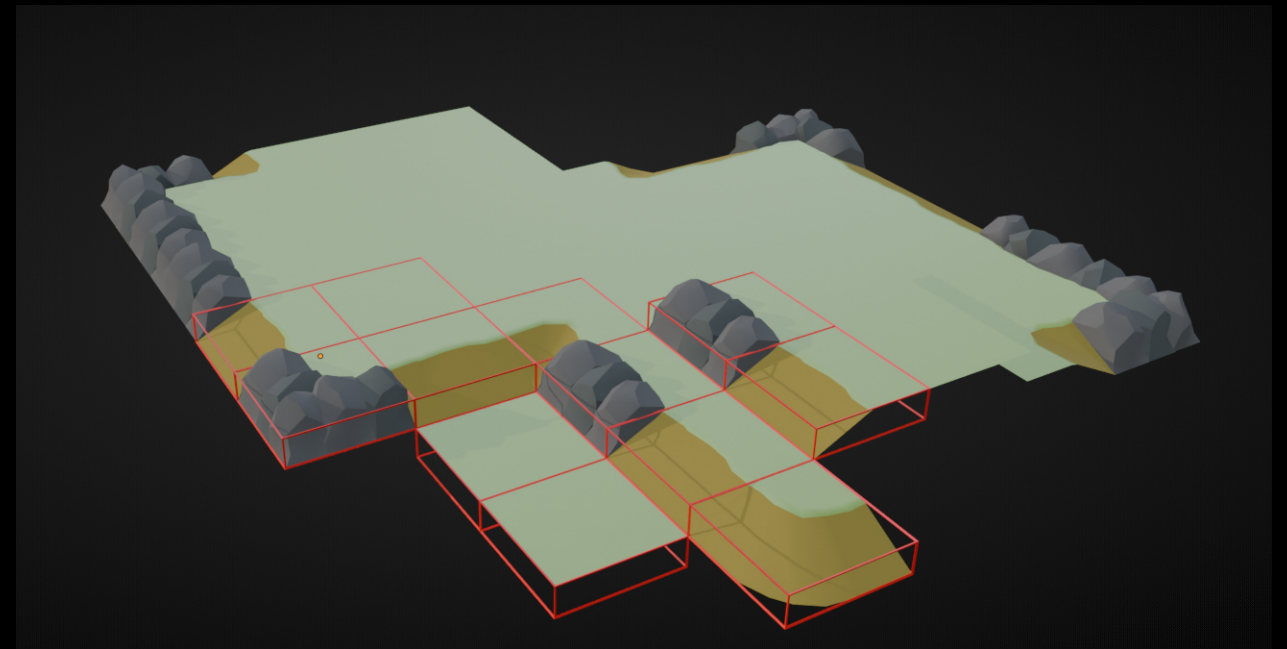
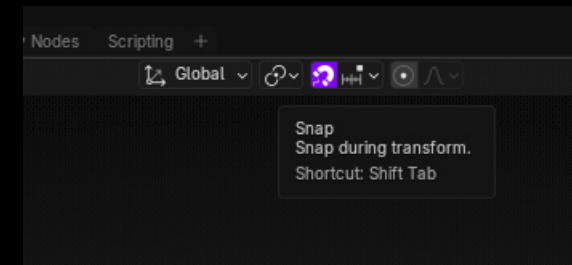
3D Models

With any tile based terrain system, we need tiles. The goal of the tiles is to create a set of joinable meshes that look seamless when they are sat in equal unit distances from each other. We can think of this in terms of a grid where each mesh is a 1 x 1 x 1 sized 3D cube, Length x Width x Height. Defining the shape and size of our boundary to determine the needs of the feel and visual of our needs. I found it more appealing to create tiles of 1 x 1 x 0.25 in units. This creates a more compact look and feels more expressive in its ability to define the environment, for a top down game perspective.

The process to create tiles begins with designing re-usable terrain pieces based on our size constraints. Create a Cube in Blender (3D Modeling Software) making its size equal to the constraints you have chosen. Add a wireframe modifier to the Cube to define the area in which you constraint your design. Ensure that the mesh, you design inside your cube, touches the edges of the wireframe so it can match pieces that will join along it's edges. Floor tiles are simple 4 vertice planes that fit along the top (height) of your boundary box. Ramps/Stairs have vertical inclines from the bottom of our boundary to the top of the boundary and then a section of flooring that meets the opposite edge.



As we build up a set of tiles that piece together into a grid like fashion. We can set Blender's snap mode on so that completed tiles can move in a snap-to-grid manner that achieves the ability to move pieces around and view our design from afar. Building up



our understanding of how we can manipulate the geometry to meet our design goals. As the pieces come together, we can begin to understand what kind of pieces we will need and what it will take to make the environment come alive.

We can continue to expand on our design, when we think about water. We can start to understand how we can paint water patterns onto the basic floor tiles.

Generate - Wfc

Once we have our tilset, we can begin to wonder how a computer can possibly take information about it and generate a map.

A popular approach to generation in games has been Wave Function Collapse (WFC). Named loosely after quantum mechanics. WFC takes a grid, think of a sudoku board, and sets all grid positions to have the possible of all tiles. Just like in sudoku where each cell in the grid is a possible value of 1 to 9.

Then the algorithm picks a starting cell and randomly selects a tile. That tile is then considered collapsed as it has found its value. We also tell the program which neighbors a cells edges can have based on how our tiles connect. So for instance, tileID 1's neighbors in the PositiveX direction can connect to tileID's 1 and 2.

Now when we collapsed the first cell it got a tileID and that tile has known possible connections in each of its 6 cubic directions. We then go through all the neighboring grid cells and remove any neighbors from their lists that don't connect with the collapsed cell. We then use the neighbor cells and make sure their neighbors can connect to their now refined list of connectable tiles.

Next we find the cell that has the smallest number of tiles it can select from. Which we know because we keep track of the number of possible connections. We then collapse that cell and iterate through the steps we did for the initial cell.

Eventually we will reach a point where the grid is filled completely, or the WFC function failed to find a solution based on our starting point. From failure, we can implement backtracking or some form of refining the failed solution to get a complete grid.

The final grid then is filled with tileID's that map to tiles and we simply instantiate them in our game world by iterating over the grid.

Complications

Wave Function Collapse may very well be a solution to the problem, yet it fails to achieve the necessary performance and simplicity desired.

Negatives:

1. Requires a lot of Preperation
2. Lacks Performance at Scale
3. Ease of Use is Complex

For these reasons, WFC has been abandoned.

Generate - Noise

Simple generation using Noise can achieve the desire of an isometric 3D tiled world terrain. This creates full control over performance, features, and artistic features that naturally feel right.

The 3D Terrain Tiles shown above, create specific constraints and limits on the type of generation required. The Tiles are isometric and have limited range in their height and rotation. Furthermore, the tiles are set to fit, like puzzle pieces, which constraints the options available for connections (placement in relation to surrounding tiles) and rotations.

This requires that the noise generates a proper solution. Random noise thus needs to reduce the resolution, posterize, then upscale back to original size. Forcing the noise into blocky square patterns. Which creates a mathematical representation that can provide usefulness to the 3D isometric tiles. We use this reduced-up-scaled-noise as our Height Map, to know the heights of tiles in a 3D space.

Then we can determine water, ramps, and cliffs on elevation changes. Filling in Ground tiles everywhere else. Finally determining the correct rotation for tiles based on surrounding tiles.